

**Московский государственный технический
университет им. Н. Э. Баумана**

Факультет «Информатика и системы управления»
Кафедра ИУ5 «Системы обработки информации и управления»

Курс «Парадигмы и конструкции языков программирования»
Отчет по лабораторной работе №3
«Работа с коллекциями на C#»

Выполнил:
Студент группы ИУ5-35Б
Гремина Анна

Проверил:
Нардид А. Н.

2025 г.

Задание

Разработать программу, реализующую работу с коллекциями.

1. Программа должна быть разработана в виде консольного приложения на языке C#.
2. Создать объекты классов «Прямоугольник», «Квадрат», «Круг».
3. Для реализации возможности сортировки геометрических фигур для класса «Геометрическая фигура» добавить реализацию интерфейса `Comparable`. Сортировка производится по площади фигуры.
4. Создать коллекцию класса `ArrayList`. Сохранить объекты в коллекцию. Отсортировать коллекцию. Вывести в цикле содержимое коллекции.
5. Создать коллекцию класса `List<Figure>`. Сохранить объекты в коллекцию. Отсортировать коллекцию. Вывести в цикле содержимое коллекции.
6. Модифицировать класс разреженной матрицы (проект `SparseMatrix`) для работы с тремя измерениями – x, y, z . Вывод элементов в методе `ToString()` осуществлять в том виде, который Вы считаете наиболее удобным. Разработать пример использования разреженной матрицы для геометрических фигур.
7. Реализовать класс «`SimpleStack`» на основе односвязного списка. Класс `SimpleStack` наследуется от класса `SimpleList` (проект `SimpleListProject`). Необходимо добавить в класс методы:
 - `public void Push(T element)` – добавление в стек;
 - `public T Pop()` – чтение с удалением из стека.
8. Пример работы класса `SimpleStack` реализовать на основе геометрических фигур.

Листинг кода

```
1  using System;
2  using System.Collections;
3  using System.Collections.Generic;
4
5  namespace Lab3Collections
6  {
7      abstract class GeometricShape : System.IComparable
8      {
9          public abstract double CalculateArea();
10
11          public override string ToString()
12          {
13              return $"{this.GetType().Name}: Площадь = {CalculateArea():F2}";
14          }
15
16          public int CompareTo(object? obj)
17          {
18              if (obj == null) return 1;
19
20              GeometricShape? other = obj as GeometricShape;
21              if (other != null)
22                  return this.CalculateArea().CompareTo(other.CalculateArea());
23              else
24                  throw new ArgumentException("Объект не является GeometricShape");
25          }
26      }
27
28      class Rectangle : GeometricShape
29      {
30          public double Width { get; set; }
31          public double Height { get; set; }
32
33          public Rectangle(double width, double height)
34          {
35              Width = width;
36              Height = height;
37          }
38      }
39  }
```

```

38
39     public override double CalculateArea()
40     {
41         return Width * Height;
42     }
43
44     public override string ToString()
45     {
46         return $"Прямоугольник ({Width} x {Height}): Площадь = {CalculateArea():F2}";
47     }
48 }
49
50 class Square : Rectangle
51 {
52     public Square(double side) : base(side, side)
53     {
54     }
55
56     public override string ToString()
57     {
58         return $"Квадрат (сторона {Width}): Площадь = {CalculateArea():F2}";
59     }
60 }
61
62 class Circle : GeometricShape
63 {
64     public double Radius { get; set; }
65
66     public Circle(double radius)
67     {
68         Radius = radius;
69     }
70
71     public override double CalculateArea()
72     {

```

```

73         return Math.PI * Radius * Radius;
74     }
75
76     public override string ToString()
77     {
78         return $"Круг (радиус {Radius}): Площадь = {CalculateArea():F2}";
79     }
80 }
81
82 class SparseMatrix
83 {
84     private Dictionary<string, double> elements;
85     private int rows;
86     private int cols;
87
88     public SparseMatrix(int rows, int cols)
89     {
90         this.rows = rows;
91         this.cols = cols;
92         elements = new Dictionary<string, double>();
93     }
94
95     public double this[int x, int y, int z]
96     {
97         get
98         {
99             string key = $"{x},{y},{z}";
100             return elements.ContainsKey(key) ? elements[key] : 0.0;
101         }
102         set
103         {
104             string key = $"{x},{y},{z}";
105             if (value != 0.0)
106                 elements[key] = value;

```

```

107         else if (elements.ContainsKey(key))
108             elements.Remove(key);
109     }
110 }
111
112 public override string ToString()
113 {
114     if (elements.Count == 0)
115         return "Разреженная матрица пуста";
116
117     string result = "Разреженная матрица (ненулевые элементы):\n";
118     foreach (var kvp in elements)
119     {
120         result += $" [{kvp.Key}] = {kvp.Value:F2}\n";
121     }
122     return result;
123 }
124 }
125
126 class SimpleStack<T>
127 {
128     private class Node
129     {
130         public T Data { get; set; }
131         public Node? Next { get; set; }
132
133         public Node(T data)
134         {
135             Data = data;
136             Next = null;
137         }
138     }

```

```

140     private Node? top;
141     private int count;
142
143     public SimpleStack()
144     {
145         top = null;
146         count = 0;
147     }
148
149     public void Push(T element)
150     {
151         Node newNode = new Node(element);
152         newNode.Next = top;
153         top = newNode;
154         count++;
155     }
156
157     public T Pop()
158     {
159         if (top == null)
160             throw new InvalidOperationException("Стек пуст");
161
162         T data = top.Data;
163         top = top.Next;
164         count--;
165         return data;
166     }
167
168     public int Count
169     {
170         get { return count; }
171     }
172
173     public bool IsEmpty
174     {

```

```

175         get { return count == 0; }
176     }
177
178     public override string ToString()
179     {
180         if (IsEmpty)
181             return "Стек пуст";
182
183         string result = "Содержимое стека (сверху вниз):\n";
184         Node? current = top;
185         while (current != null)
186         {
187             result += $" {current.Data}\n";
188             current = current.Next;
189         }
190         return result;
191     }
192 }
193
194 class Program
195 {
196     static void Main(string[] args)
197     {
198         Console.WriteLine("=== Лабораторная работа №3: Коллекции ===\n");
199
200         // 2. Создание объектов
201         Rectangle rect1 = new Rectangle(5, 10);
202         Rectangle rect2 = new Rectangle(3, 7);
203         Square square1 = new Square(6);
204         Square square2 = new Square(4);
205         Circle circle1 = new Circle(5);
206         Circle circle2 = new Circle(3);
207
208         // 3. ArrayList с сортировкой
209         Console.WriteLine("--- 3. ArrayList с сортировкой ---");
210         ArrayList arrayList = new ArrayList();
211         arrayList.Add(rect1);
212         arrayList.Add(square1);
213         arrayList.Add(circle1);
214         arrayList.Add(rect2);
215         arrayList.Add(square2);
216         arrayList.Add(circle2);
217
218         Console.WriteLine("До сортировки:");
219         foreach (GeometricShape shape in arrayList)
220             Console.WriteLine($" {shape}");
221
222         arrayList.Sort();
223
224         Console.WriteLine("\nПосле сортировки по площади:");
225         foreach (GeometricShape shape in arrayList)
226             Console.WriteLine($" {shape}");
227
228         // 4. List<GeometricShape>
229         Console.WriteLine("\n--- 4. List<GeometricShape> ---");
230         List<GeometricShape> shapeList = new List<GeometricShape>();
231         shapeList.Add(new Rectangle(8, 4));
232         shapeList.Add(new Square(5));
233         shapeList.Add(new Circle(4));
234
235         Console.WriteLine("До сортировки:");
236         foreach (var shape in shapeList)
237             Console.WriteLine($" {shape}");
238
239         shapeList.Sort();
240
241         Console.WriteLine("\nПосле сортировки по площади:");
242         foreach (var shape in shapeList)

```

```

243         Console.WriteLine($" {shape}");
244
245         // 6. Разреженная матрица
246         Console.WriteLine("\n--- 6. Разреженная матрица (3D) ---");
247         SparseMatrix matrix = new SparseMatrix(100, 100);
248
249         // Пример использования для геометрических фигур
250         matrix[0, 0, 0] = rect1.CalculateArea();
251         matrix[1, 2, 3] = circle1.CalculateArea();
252         matrix[5, 10, 15] = square1.CalculateArea();
253         matrix[10, 20, 30] = 42.5;
254
255         Console.WriteLine(matrix.ToString());
256
257         // 7-8. SimpleStack на основе геометрических фигур
258         Console.WriteLine("\n--- 7-8. SimpleStack<GeometricShape> ---");
259         SimpleStack<GeometricShape> stack = new SimpleStack<GeometricShape>();
260
261         Console.WriteLine("Добавляем фигуры в стек:");
262         stack.Push(new Rectangle(3, 4));
263         Console.WriteLine(" Добавлен прямоугольник 3x4");
264         stack.Push(new Square(5));
265         Console.WriteLine(" Добавлен квадрат со стороной 5");
266         stack.Push(new Circle(2));
267         Console.WriteLine(" Добавлен круг с радиусом 2");
268
269         Console.WriteLine($"Всего элементов в стеке: {stack.Count}");
270         Console.WriteLine(stack.ToString());
271
272         Console.WriteLine("Извлекаем элементы из стека:");
273         while (!stack.IsEmpty)
274         {
275             GeometricShape shape = stack.Pop();
276             Console.WriteLine($" Извлечено: {shape}");
277         }
278
279         Console.WriteLine($"Всего элементов в стеке: {stack.Count}");
280
281         Console.WriteLine("\n--- Пример работы стека с разными типами ---");
282         SimpleStack<GeometricShape> demoStack = new SimpleStack<GeometricShape>();
283
284         demoStack.Push(rect1);
285         demoStack.Push(square2);
286         demoStack.Push(circle2);
287
288         Console.WriteLine("Демонстрация стека:");
289         Console.WriteLine(demoStack.ToString());
290
291         Console.WriteLine("\nНажмите любую клавишу для выхода...");
292         Console.ReadKey();
293     }
294 }
295 }

```

Результат работы программы

```
=== Лабораторная работа №3: Коллекции ===
```

```
--- 3. ArrayList с сортировкой ---
```

До сортировки:

Прямоугольник (5 x 10): Площадь = 50.00

Квадрат (сторона 6): Площадь = 36.00

Круг (радиус 5): Площадь = 78.54

Прямоугольник (3 x 7): Площадь = 21.00

Квадрат (сторона 4): Площадь = 16.00

Круг (радиус 3): Площадь = 28.27

После сортировки по площади:

Квадрат (сторона 4): Площадь = 16.00

Прямоугольник (3 x 7): Площадь = 21.00

Круг (радиус 3): Площадь = 28.27

Квадрат (сторона 6): Площадь = 36.00

Прямоугольник (5 x 10): Площадь = 50.00

Круг (радиус 5): Площадь = 78.54

```
--- 4. List<GeometricShape> ---
```

До сортировки:

Прямоугольник (8 x 4): Площадь = 32.00

Квадрат (сторона 5): Площадь = 25.00

Круг (радиус 4): Площадь = 50.27

После сортировки по площади:

Квадрат (сторона 5): Площадь = 25.00

Прямоугольник (8 x 4): Площадь = 32.00

Круг (радиус 4): Площадь = 50.27

```
--- 6. Разреженная матрица (3D) ---
```

Разреженная матрица (ненулевые элементы):

[0,0,0] = 50.00

[1,2,3] = 78.54

[5,10,15] = 36.00

[10,20,30] = 42.50

--- 7-8. SimpleStack<GeometricShape> ---

Добавляем фигуры в стек:

Добавлен прямоугольник 3x4

Добавлен квадрат со стороной 5

Добавлен круг с радиусом 2

Всего элементов в стеке: 3

Содержимое стека (сверху вниз):

Круг (радиус 2): Площадь = 12.57

Квадрат (сторона 5): Площадь = 25.00

Прямоугольник (3 x 4): Площадь = 12.00

Извлекаем элементы из стека:

Извлечено: Круг (радиус 2): Площадь = 12.57

Извлечено: Квадрат (сторона 5): Площадь = 25.00

Извлечено: Прямоугольник (3 x 4): Площадь = 12.00

Всего элементов в стеке: 0

--- Пример работы стека с разными типами ---

Демонстрация стека:

Содержимое стека (сверху вниз):

Круг (радиус 3): Площадь = 28.27

Квадрат (сторона 4): Площадь = 16.00

Прямоугольник (5 x 10): Площадь = 50.00

Нажмите любую клавишу для выхода...